FlagMaster

Documentazione di Progetto
Programmazione Avanzata

Luca Maffioli
Matricola: 677450
A.A. 2025/2026

Indice

1 Caso d'uso **2**

2 Client **3**
2.1 Grafica 3
2.1.1 Login e registrazione 3
2.1.2 Home 4
2.1.3 My Lists e Edit List 5
2.1.4 Gioco (Quiz) 6
2.1.5 Statistiche 8
2.2 Struttura del codice 9
2.3 Principali scelte progettuali 9

3 Server **10**
3.1 API 10
3.2 Struttura 10
3.3 Principali scelte progettuali 11
3.4 Database 11

4 Unit Test **11**

5 Utilizzo AI **12**

1 Caso d'uso

FlagMaster è un *Quiz Game* in cui l'utente deve associare ad una bandiera il nome del paese corrispondente scegliendo fra diverse opzioni. Per rendere il gioco più interessante e personalizzabile è possibile creare delle liste di paesi sulle quali testare le proprie abilità. Per ogni lista vengono mostrate delle statistiche che permettono all'utente di tenere traccia dei risultati ottenuti.

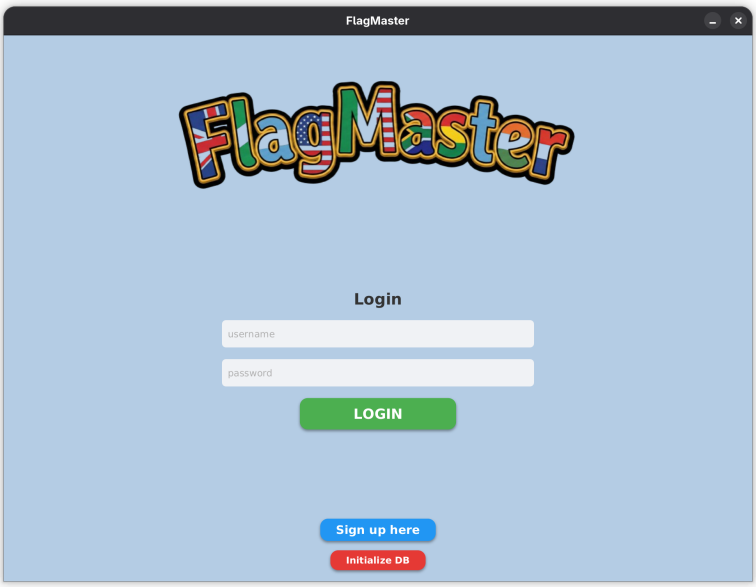
2 Client

2.1 Grafica

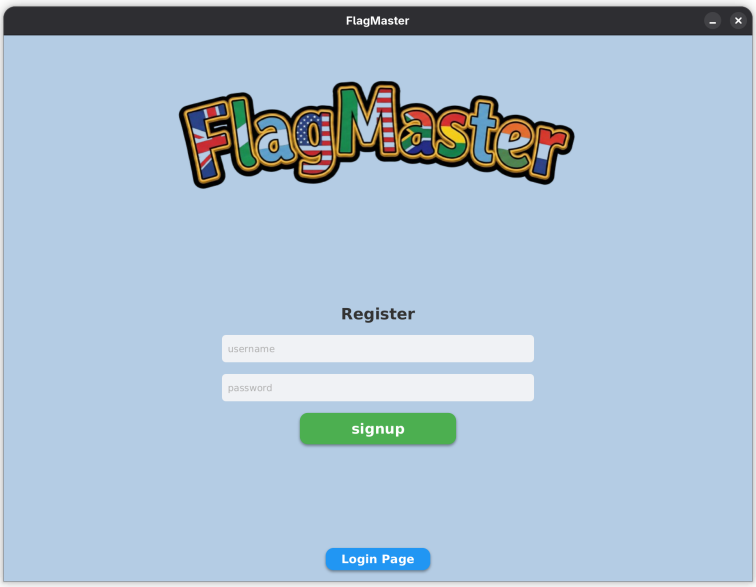
Di seguito sono riportate le schermate dell'applicazione.

2.1.1 Login e registrazione

Una volta avviata l'applicazione, dopo aver inizializzato il database, è possibile accedere o creare un nuovo account.



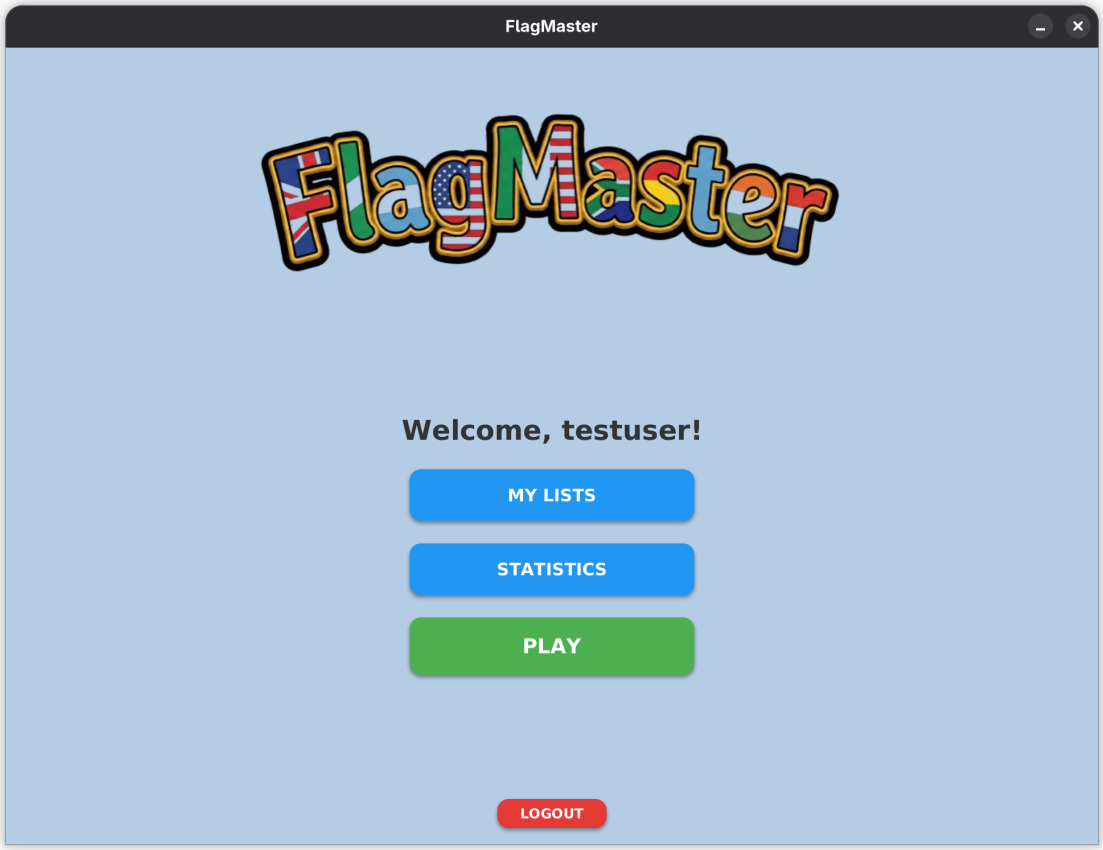
(a) Schermata di Login



(b) Schermata di Registrazione

2.1.2 Home

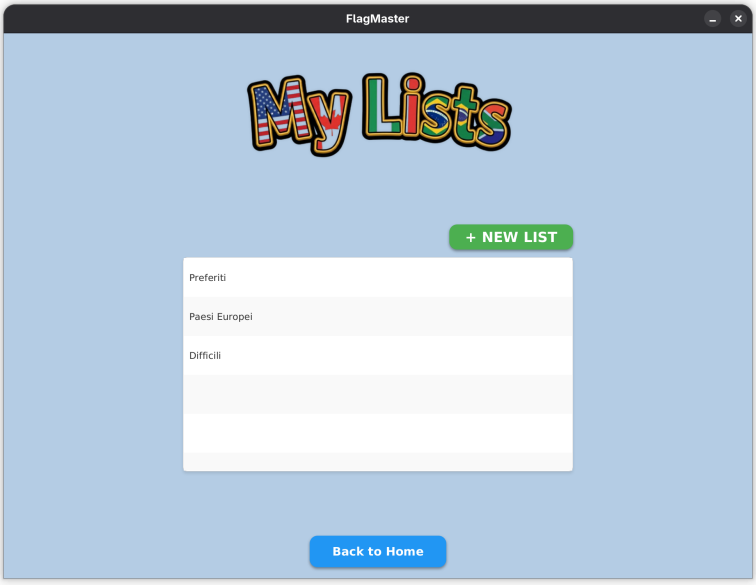
La home è il punto centrale dell'applicazione. Da qui si accede alle proprie liste personali, alle statistiche o si avvia una nuova partita.



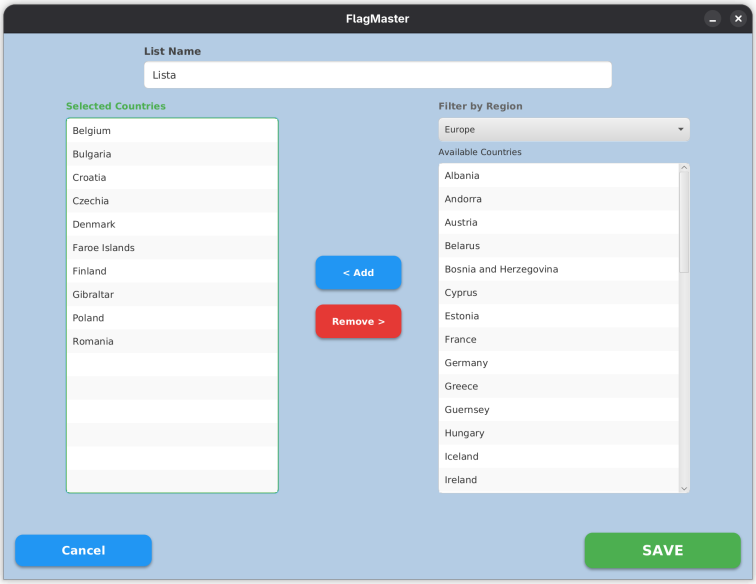
2.1.3 My Lists e Edit List

Grazie a questa sezione è possibile creare liste di paesi personalizzate sulle quali testare le proprie conoscenze tramite il quiz game. Dalla schermata "MyLists" è possibile creare una nuova lista o modificarne una già presente (doppio click o tasto destro per apertura menù sugli elementi). Al momento della registrazione vengono create tre liste "Start" in modo che l'utente possa giocare subito.

La creazione e la modifica avvengono tramite la stessa schermata in cui è possibile selezionare paesi dalla lista di destra e aggiungerli con il tasto Add, o selezionare dalla lista di sinistra ed eliminarli con il tasto Remove.



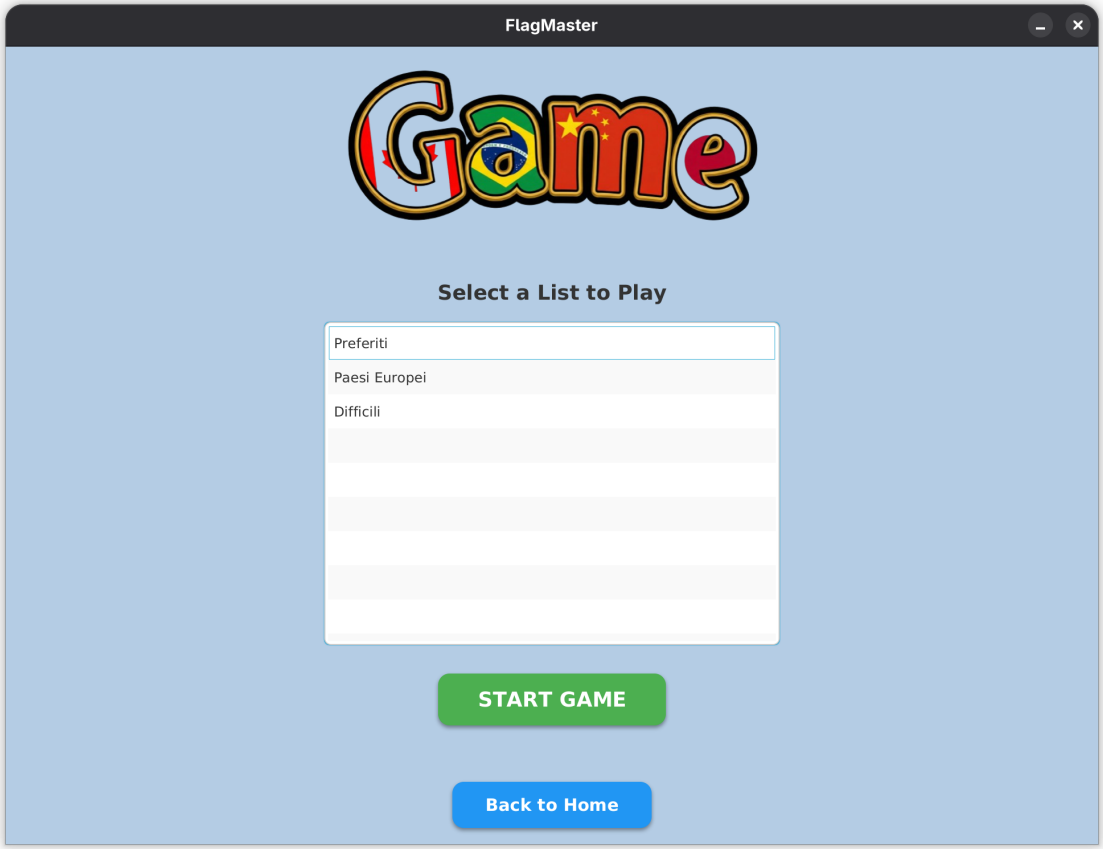
(a) Elenco liste create



(b) Editor lista

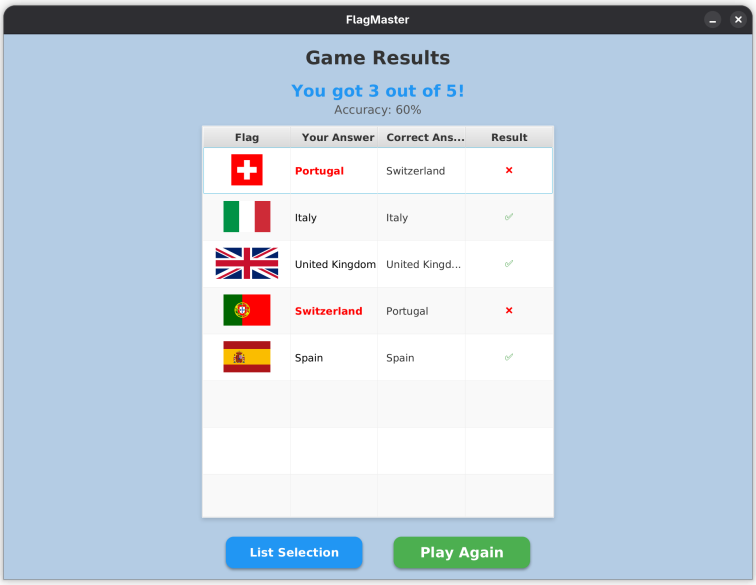
2.1.4 Gioco (Quiz)

Prima di poter giocare è necessario selezionare la lista dalla quale verranno create le domande.



Seguono diverse domande, ciascuna a risposta multipla. Alla fine del quiz è presente una schermata di resoconto per dare all'utente un feedback immediato.

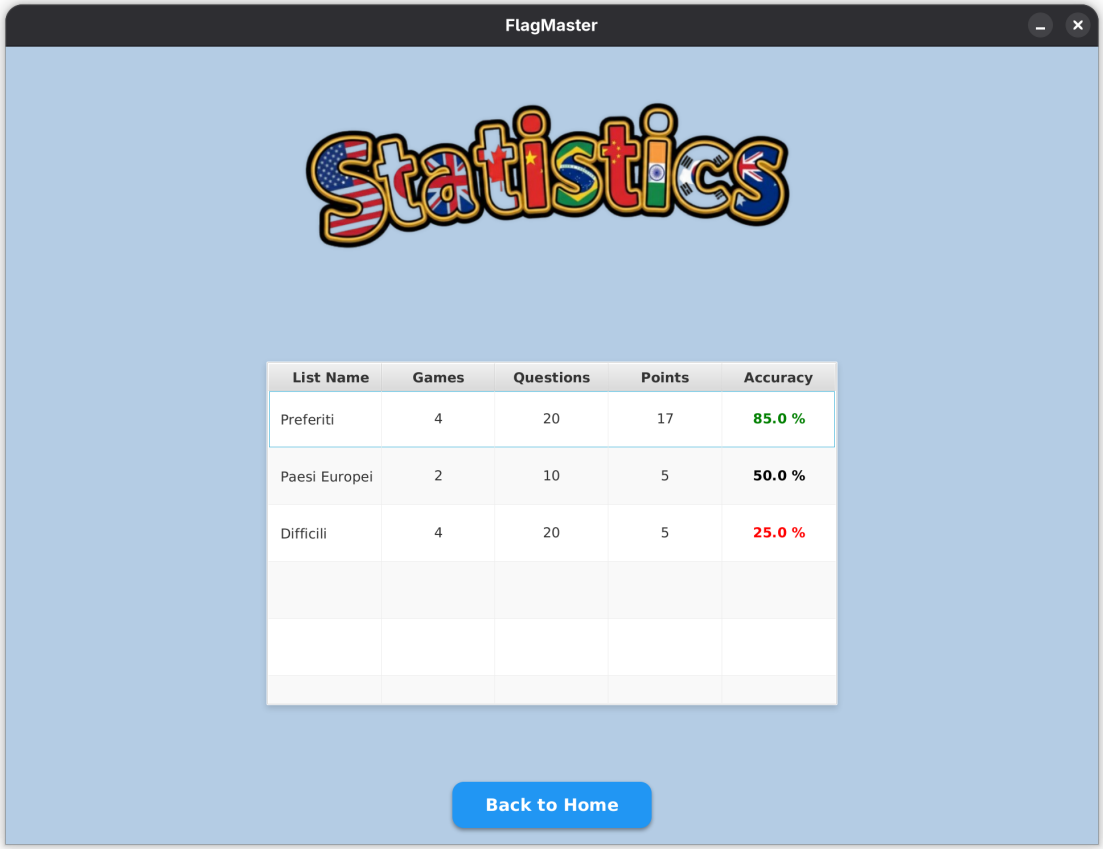
(a) Schermata di gioco



(b) Risultati Finali

2.1.5 Statistiche

Dalla schermata dedicata è possibile visualizzare alcuni dati relativi ai quiz effettuati sulle liste.



2.2 Struttura del codice

Organizzazione del progetto e delle componenti principali, non sono presenti tutte le classi ma solo alcune a scopo illustrativo.

```
Applicazione
├── src/main/java/it/unipi/applicazione
│   ├── controller ............... Gestione interazione tra interfaccia e logica
│   │   ├── EditListController.java
│   │   └── GameController.java
│   ├── model ....................................................DTO e oggetti
│   ├── service ........................... Implementazione richieste al server
│   │   └── APIclient.java
│   └── utils ................................................ Classi di utilità
│       └── Session.java
├── src/main/resources/it/unipi/applicazione
│   ├── fxml ................................File descrittivi delle interfacce
│   ├── style.css ................................ Descrizione stile elementi
│   └── img ......................................................Loghi e scritte
└── src/main/resources/it
    └── log4j2.xml ..................................................... logger
```

2.3 Principali scelte progettuali

La gestione dello stato lato client è affidata alla classe `Session` (implementata come *Singleton*), che funge da archivio centrale per le informazioni dell'utente loggato. In particolare essa memorizza il **token di autenticazione** essenziale per autorizzare le richieste. Inoltre la classe mantiene i dati necessari durante la navigazione tra le schermate, come la lista attualmente selezionata o i risultati parziali.

Il fulcro della comunicazione con il server è rappresentato dalla classe `ApiClient`. Questa componente centralizza la logica di rete implementando metodi generici per le richieste `GET` e `POST`: si occupa della costruzione della richiesta (inclusa l'iniezione del token nel campo `Authorization` dell'header http), del parsing dei payload JSON e della gestione degli errori HTTP. Le altre classi *Service* estendono `ApiClient`, ereditandone le funzionalità base e invocando i metodi specifici per ogni endpoint, evitando così la duplicazione del codice.

L'architettura complessiva segue la logica *Thin Client*: l'elaborazione dell'applicazione è ridotta al minimo e si delega la complessità al server. Un esempio concreto è il flusso di avvio di una partita: il client richiede i dati e il server risponde con una lista di oggetti `QuestionDTO` già strutturati. Questi oggetti contengono tutte le informazioni pronte all'uso (domanda, opzioni, risposta corretta), permettendo all'interfaccia grafica di limitarsi alla sola visualizzazione dei dati senza ulteriori elaborazioni.

3 Server

3.1 API

Di seguito sono elencati i principali endpoint esposti dal server.

Metodo	Endpoint	Descrizione
POST	/api/setup/inizializza	Inizializzazione DB.
POST	/api/auth/login	Effettua login giocatore.
POST	/api/auth/logout	Logout (eliminazione token).
POST	/api/auth/signup	Crea nuovo account.
GET	/api/list/my-lists	Scarica tutte le liste del giocatore.
POST	/api/list/update	Salva o aggiorna lista paesi.
POST	/api/list/delete/{listId}	Elimina lista.
GET	/api/country/regions	Scarica tutti i continenti.
GET	/api/country/countries/{region}	Scarica paesi del continente.
POST	/api/score/add	Salva il punteggio della partita.
GET	/api/score/stats	Scarica le statistiche del giocatore.
GET	/api/game/new/{listId}	Scarica una nuova partita sulla lista.

3.2 Struttura

Organizzazione del progetto e delle componenti principali, non sono presenti tutte le classi ma solo alcune a scopo illustrativo.

```
Server
└── src/main/java/it/unipi/server
    ├── config ..........Interceptor e configurazione per gestione token utente
    ├── controller ..............................................Endpoint REST
    │   ├── AuthController.java
    │   └── GameController.java
    ├── dto ............................................ Data transfer object
    ├── model ........................................... Entità mappate sul DB
    ├── repository ................................... Interfacce JPA per MySQL
    │   └── CountryRepository.java
    ├── service ....................................Logica e trasferimento dati
    │   └── UserListService.java
    ├── utils
    │   └── Crypto.java ............................Gestione hashing password
```

3.3 Principali scelte progettuali

Per l'interazione con il database è stato utilizzato **Spring Data JPA**. Questo permette di astrarre completamente la complessità del linguaggio SQL, mappando le tabelle del database direttamente sulle classi Java (Entity). L'accesso ai dati avviene grazie al **Repository**: invece di gestire connessioni JDBC di basso livello, sono state definite delle interfacce che estendono `CrudRepository`, delegando al framework la creazione automatica delle query standard. Per query più complesse è fatto ricorso all'annotazione `@Query`, che ha permesso di definire esplicitamente l'interrogazione del db direttamente all'interno dell'interfaccia.

Per gestire la multi utenza in modo sicuro è stato usato un token di sessione associato all'utente durante il login. L'architettura implementa il pattern *Interceptor* di Spring Boot: tramite le classi `AuthInterceptor` e `WebConfig`, il sistema intercetta le richieste HTTP in ingresso. Questo permette di verificare la validità del token per tutti gli endpoint (esclusi quelli in cui il token non è significativo), centralizzando la logica e mantenendo i *Controller* e i *Service* focalizzati esclusivamente sulla logica specifica.

Infine, la procedura di popolamento iniziale del database (endpoint `/api/initializeDB`) è implementata nel `SetupService`. Questa funzionalità esegue un reset dell'ambiente, svuotando le tabelle esistenti e ricaricando il catalogo dei paesi prelevando dati aggiornati dall'API pubblica **REST Countries**; il parsing dei dati JSON ricevuti è gestito tramite la libreria esterna **Gson**.

3.4 Database

I parametri di connessione al database (URL JDBC, username e password) sono configurati nel file `application.properties` situato nelle risorse del progetto server.

- **User**: Registro degli utenti.
- **Country**: Catalogo statico dei paesi, popolato tramite API esterna (*RestCountries*) in fase di inizializzazione.
- **UserList**: Rappresenta le liste personalizzate. Collega ogni lista al proprio creatore, aggregando i paesi selezionati e gestendo, tramite meccanismi a cascata, i punteggi associati.
- **Score**: Risultati delle partite. Storico delle partite. Ogni punteggio è associato a un utente e alla lista su cui è stata giocata la partita.

4 Unit Test

Sono stati implementati alcuni Unit Tests per la classe UserListService, responsabile della gestione delle liste personalizzate. Utilizzando Mockito, è stato possibile simulare il comportamento dei repository (UserListRepository e CountryRepository), isolando la logica dalle interazioni reali con il database. I test verificano che le operazioni di recupero, eliminazione e soprattutto di aggiornamento vengano eseguite correttamente; in particolare, per l'update, si controlla tramite asserzioni che l'algoritmo gestisca in modo

coerente l'aggiunta e la rimozione dei paesi dalla lista, confermando l'integrità dei dati prima del salvataggio.

5 Utilizzo AI

L'implementazione è stata supportata dall'uso di **Gemini 3 Pro**, utilizzato sia per la generazione e lo styling dei file `FXML`, sia come ausilio nella risoluzione degli errori. Il modello è inoltre stato utilizzato come motore di ricerca per aspetti specifici non visti a lezione.